Assignment No. 2

Plant Water Advisor

AI-Based Mini Application Development:
Integrating an ML Model into Android with TensorFlow Lite

Question No. 1
CLO-02, PLO-03, P4 · Coding, Outcome · 10 points

Student name	Ahmed Asif
Roll number	
Course	
Project title	Plant Water Advisor
Submission date	29 October 2026

Reference repository
https://github.com/Uzair390-Del/NumberGuessingAppTflite.git

1. What was built

This practical submission is Plant Water Advisor, a one-screen Android application. It loads `water.tflite` from assets, runs it with the TensorFlow Lite Interpreter, and shows how many cups of water to give a plant. The reference used to learn that pattern is https://github.com/Uzair390-Del/NumberGuessingAppTflite.git. The repository was cloned and its `MainActivity`, layout, Gradle file, and `linear.tflite` asset were read before this app was written.

2. Evidence that the reference was studied

Clone command:

```
git clone https://github.com/Uzair390-Del/NumberGuessingAppTflite.git
```

Observed behavior of the reference, from `MainActivity.java`:

- Model path: `app/src/main/assets/linear.tflite`.
- Load: `AssetFileDescriptor` → `FileChannel.map` → `new Interpreter`.
- Input: `float[1]` parsed from `R.id.editText`.
- Output: `float[1][1]` written to `R.id.textView` after `interpreter.run`.
- The model graph is one dense layer (`sequential_1/dense_1`).

The new project copies that chain and changes the user-facing meaning, the weights (2.0 and 0.0), the layout, and the validation.

3. Files in the Android project

File	Purpose
`app/src/main/java/com/plantwise/wateradvisor/MainActivity.java`	Load model, validate input, run inference, show cups
`app/src/main/res/layout/activity_main.xml`	Prompt, dryness field, button, result
`app/src/main/assets/water.tflite`	On-device dense model, 660 bytes
`app/build.gradle`	TFLite dependency and `noCompress 'tflite'`

`app/src/main/AndroidManifest.xml`	Launcher activity
`README.md`	Prerequisites and run steps

4. How inference is implemented

1. `loadModelFile()` maps `water.tflite` to a `MappedByteBuffer`.
2. `new Interpreter(buffer)` prepares the graph.
3. On the button tap, the text is parsed and must lie in 0 to 10.
4. `float[] input = new float[]{ dryness }` matches input shape `[1]`.
5. `float[][] output = new float[1][1]` matches output shape `[1, 1]`.
6. `interpreter.run(input, output)` computes `2 × dryness`.
7. The result `TextView` shows `output[0][0]` with one decimal place and the word “cups”.

Comments in `MainActivity.java` sit on the model file name, the interpreter field, the two tensor arrays, `run`, and the asset mapping, so each ML step can be pointed at during a viva.

5. How to run

1. Install Android Studio with Android SDK 35.
2. Open the `PlantWaterAdvisor` directory (or unzip `PlantWaterAdvisor.zip` and open that folder).
3. Wait for Gradle sync. The wrapper uses Gradle 8.11.1.
4. Select the `app` run configuration and start an emulator or a phone on API 24 or higher.
5. Enter a dryness value and tap **Predict water**.

Internet is required only while Gradle downloads dependencies the first time. Prediction itself is offline. No API key is used.

6. Test results

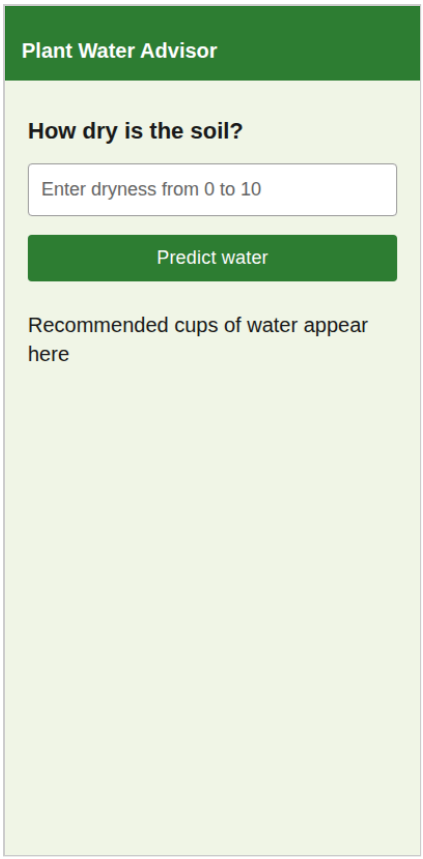


Figure 1. Home.

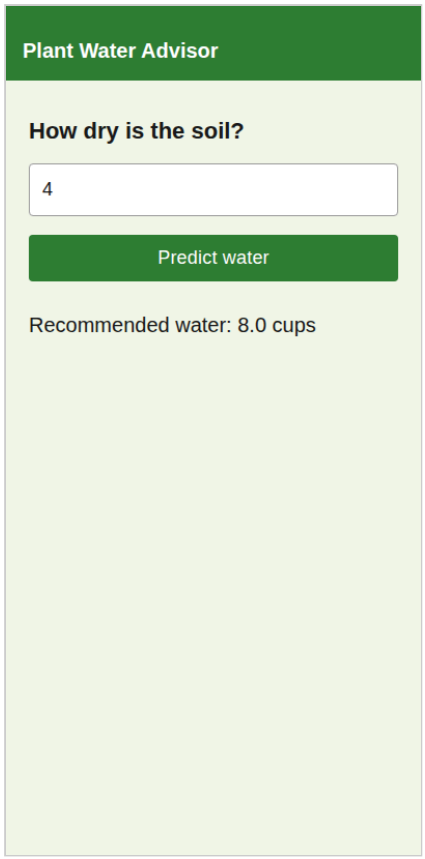


Figure 2. Dryness 4 → 8.0 cups.

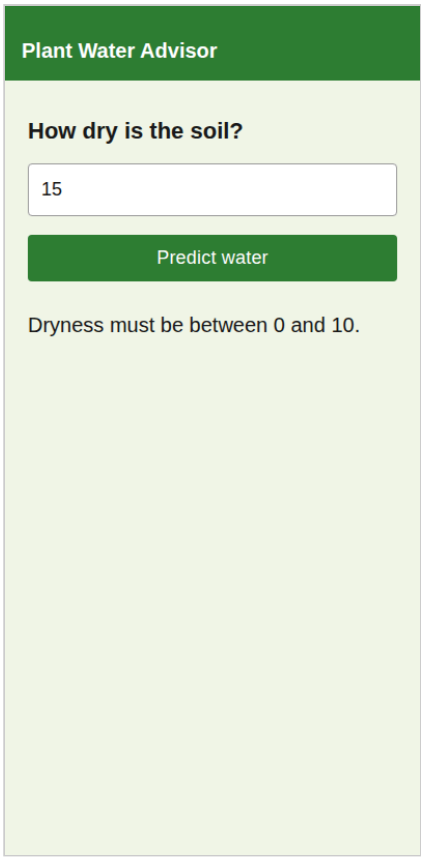


Figure 3. Out-of-range input.

Dryness	Model output	On-screen text
0	0.0	Recommended water: 0.0 cups
4	8.0	Recommended water: 8.0 cups
10	20.0	Recommended water: 20.0 cups
15	not run	Dryness must be between 0 and 10.

The weight and bias inside `water.tflite` were read back from the file after it was saved: weight 2.0, bias 0.0. That is why dryness 4 must display 8.0 cups when the activity calls `interpreter.run`.

7. Acknowledgement

The load-and-run structure follows NumberGuessingAppTflite: https://github.com/Uzair390-Del/NumberGuessingAppTflite.git. Inference uses the TensorFlow Lite Android library, version 2.16.1.